

```

test\resources
[INFO]
[INFO] --- maven-compiler-
plugin:3.1:testCompile (default-
testCompile) @ devops-app ---
[INFO] Nothing to compile - all classes
are up to date
[INFO]
[INFO] --- maven-surefire-
plugin:2.12.4:test (default-test) @
devops-app ---
[INFO] No tests to run.
[INFO]
[INFO] --- maven-war-plugin:2.2:war
(default-war) @ devops-app ---
[INFO] Packaging webapp
[INFO] Assembling webapp [devops-app]
in [C:\Users\Mitesh\eclipse-workspace-
new\devops-app\target\devops-app]
[INFO] Processing war project
[INFO] Copying webapp resources [C:\
Users\Mitesh\eclipse-workspace-new\
devops-app\src\main\webapp]
[INFO] Webapp assembled in [59 msecs]
[INFO] Building war: C:\Users\Mitesh\
eclipse-workspace-new\devops-app\
target\devops-app.war
[INFO] WEB-INF\web.xml already added,
skipping
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 6.639 s
[INFO] Finished at: 2019-10-
26T20:09:13+05:30
[INFO] -----
Picked up _JAVA_OPTIONS: -Xmx512M

```

There are no test resources available as of now. Hence, test results won't be available. Verify the *Packaging WebApp* step. As it is a Web application, it will create a WAR file.

Servers in Eclipse

Apache Tomcat (or just Tomcat) is one of the most popular open source (Apache Licence 2.0) implementations of the Java Servlet and JavaServer Pages technologies. It provides a 'pure Java' environment in which Java code can execute. It is very easy to use and,

hence, very popular in project teams.

We can verify the WAR file created in the earlier section by deploying it into the Tomcat server.

Step 1: Click on the *Servers* tab in the Eclipse window. You will see the message: 'No servers are available. Click this link to create a new server...'

Step 2: We want to configure the Tomcat server. Hence, select *Apache*. Select *Tomcat v7.0 Server*.

Step 3: Provide the server's host name and name. Click on *Next*.

Step 4: Specify the Tomcat installation directory. Click on *Next*.

Step 5: Select the newly created project as resources that are configured on the server, and click on *Add*.

Step 6: Click on *Finish*.

Step 7: Right click on the newly added server and select *Start*.

Step 8: Right click on our *Project* -> *Run As* -> *Run on Server* -> *Choose an existing Server*. Select *Tomcat v7.0 server at Localhost* that we created. Click on *Finish*.

This action will open the application page in Eclipse browser at <http://localhost:9999/devops-app/>.

What if we want to publish changes directly to the server and not manually deploy it every time we change files? Take the following steps for that.

Step 1: Right click on the Tomcat server in the *Servers* tab and select *Open* or press *F3*.

Step 2: This will open the *Overview* tab. Go to the publishing section and select *Automatically publish when resources change* and *Save*.

Step 3: Using the browser, visit <http://localhost:9999/devops-app/> to verify if the changes have taken place.

Eclipse and Java programming

Next, let's try and create a simple Java program.

Right click on the *src* folder in Eclipse and create a new class *TableTransformer*.

Select method stubs as shown in Figure 1, and click on *Finish*. The

following code will be generated:

```

public class TableTransformer {

    public TableTransformer() {
        // TODO Auto-generated
        constructor stub
    }

    public static void main(String[]
args) {
        // TODO Auto-generated method
        stub
    }
}

```

Template proposals help to write programs quickly. Use *Ctrl + space* in the IDE work area to bring the template proposals.

Select a proposal and click on it to bring the code block in the program, as shown below:

```

public class TableTransformer {

    public TableTransformer() {
        // TODO Auto-generated
        constructor stub
    }

    public static void main(String[]
args) {
        // TODO Auto-generated method
        stub

        if (condition) {
            try {

            } catch (Exception e) {
                // TODO: handle
                exception
            }
            } else {

            try {

            } finally {
                // TODO: handle finally clause
            }
        }
    }
}

```